

Module : Kotlin (M202)

Devoir N° 2

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM 201 - DEVOAM 202 Niveau : 2ème année

Exercice 1 :

Créer un système de gestion de commandes dans un restaurant utilisant les coroutines de Kotlin pour traiter les commandes, la préparation des plats, et la gestion des paiements de manière asynchrone.

Partie 1 : Prise de Commandes

→ Classe Commande :

◆ Propriétés :

- id : Int (ID unique de la commande)
- plats : List<String> (liste des plats commandés)
- total : Double (montant total de la commande)

◆ Constructeur qui initialise les propriétés.

◆ Méthode afficherDetails() : affiche les détails de la commande.

→ Classe Serveur :

◆ Propriétés :

- nom : String

◆ Méthodes :

- prendreCommande(plats: List<String>, prix: List<Double>) :
Commande : permet au serveur de prendre une commande.
- afficherCommande(commandes: List<Commande>) : affiche toutes les commandes prises par le serveur.

Partie 2 : Préparation des Plats

→ Classe Cuisinier :

◆ Propriétés :

- nom : String

◆ Méthodes :

- preparerPlat(plat: String): String : simule la préparation d'un plat en quelques secondes (avec un délai simulé).
- preparerCommande(commande: Commande): List<String> : prépare tous les plats d'une commande de manière asynchrone (avec async et await).

→ **Classe Cuisine :**

◆ Propriétés :

- cuisiniers : List<Cuisinier>

◆ Méthode :

- gererPreparationCommande(commande: Commande) : distribue les plats d'une commande entre plusieurs cuisiniers pour les préparer simultanément (avec des coroutines).

Partie 3 : Paiements

5. **Classe Caisse :**

○ Méthodes :

- traiterPaiement(commande: Commande): Boolean : simule un paiement asynchrone avec une chance d'échec (à gérer avec des exceptions).
- annulerPaiement(commande: Commande) : permet d'annuler un paiement (en annulant la coroutine associée).

Partie 4 : Gestion des Commandes avec Coroutines

→ **Classe Restaurant :**

◆ Propriétés :

- commandes : MutableList<Commande>

- serveurs : List<Serveur>
- cuisine : Cuisine
- caisse : Caisse

◆ Méthodes :

- prendreCommandeEtTraiter(serveur: Serveur, plats: List<String>, prix: List<Double>) : permet à un serveur de prendre une commande, de la préparer et de la facturer de manière asynchrone.
- afficherCommandesEnCours() : affiche les commandes en cours de traitement.

Questions :

1. Le serveur prend une commande avec une liste de plats et les prix associés.
2. La cuisine reçoit la commande et distribue la préparation des plats entre plusieurs cuisiniers, qui les préparent simultanément (utilisation de async et await pour paralléliser).
3. Une fois la commande prête, elle est envoyée à la caisse pour être facturée.
4. Le paiement est traité de manière asynchrone avec un risque d'échec (gestion d'exceptions).
5. Le système affiche toutes les commandes en cours et gère l'annulation si nécessaire (par exemple, si le client annule sa commande).

Exercice 2 :

Créer un système de gestion de stock pour un entrepôt en utilisant les **coroutines Kotlin**. Ce système doit gérer l'ajout et le retrait de produits de manière asynchrone, tout en offrant la possibilité d'exécuter des inventaires et de traiter des commandes clients en parallèle. Les différentes opérations sur le stock doivent être réalisées de manière non bloquante afin d'optimiser l'utilisation des ressources.

Partie 1 : Produits et Stock

→ Classe Produit :

◆ Propriétés :

- id : Int (ID unique du produit)
- nom : String
- quantite : Int (quantité actuelle en stock)

◆ Constructeur pour initialiser les propriétés.

◆ Méthode afficherDetails() pour afficher les informations du produit.

→ Classe Stock :

◆ Propriétés :

- produits : MutableMap<Int, Produit> (Map associant les ID de produits à leur objet Produit respectif)

◆ Méthodes :

- ajouterProduit(produit: Produit) : ajoute un nouveau produit au stock.
- ajouterQuantite(idProduit: Int, quantite: Int) : ajoute de la quantité à un produit déjà existant (de manière asynchrone).
- retirerQuantite(idProduit: Int, quantite: Int) : retire de la quantité à un produit (avec gestion des erreurs si la quantité demandée est trop grande).
- afficherStock() : affiche la liste des produits avec leur quantité actuelle.

Partie 2 : Commandes Clients

→ Classe Commande :

◆ Propriétés :

- idCommande : Int
- produits : List<Pair<Int, Int>> (liste de paires représentant un produit et la quantité commandée)

◆ Méthodes :

- `afficherCommande()` : affiche les détails de la commande (ID produit et quantité demandée).

→ Classe **GestionnaireCommandes** :

◆ Propriétés :

- `stock` : Stock
- `commandes` : `MutableList<Commande>`

◆ Méthodes :

- `traiterCommande(commande: Commande)` : traite une commande en vérifiant la disponibilité des produits en stock (asynchrone avec des coroutines).
- `gererCommandes(commandes: List<Commande>)` : permet de traiter plusieurs commandes en parallèle (avec `launch` pour chaque commande).

Partie 3 : Opérations Asynchrones sur le Stock

→ Classe **Entrepot** :

◆ Propriétés :

- `stock` : Stock
- `gestionnaireCommandes` : `GestionnaireCommandes`

◆ Méthodes :

- `ajouterProduitAuStock(produit: Produit)` : ajoute un produit dans le stock (asynchrone avec `launch`).
- `retirerProduitDuStock(idProduit: Int, quantite: Int)` : retire une certaine quantité de stock (asynchrone avec gestion d'exceptions si le stock est insuffisant).
- `gererInventaire()` : effectue un inventaire complet des produits en stock (exécuté en parallèle avec d'autres tâches).

Partie 4 : Flux d'Événements

→ Gestion des Événements avec Flow :

- ◆ Utilisez Flow pour gérer les événements de mise à jour du stock en temps réel. Chaque ajout ou retrait de produit génère un événement qui est capturé par un Flow et traité par l'application pour mettre à jour l'inventaire.

Questions :

1. Ajouter des produits au stock avec des quantités initiales.
2. Retirer ou ajouter des quantités de produits de manière asynchrone.
3. Traiter plusieurs commandes clients en parallèle.
4. Gérer un inventaire en temps réel à l'aide des Flow pour capturer les événements de mise à jour du stock.